

A Report on DJ Application Development Using Object Oriented Programming Concepts

Bruno Bucalon Serra
Department of Computer Science
University of London

CONTENTS

I	Introduction	1
II	Implementation Logic	1
II-A	Loading, Playback and Mixing	1
II-B	Music Library and Persistence	2
II-C	Interactive Hot Cues	2
II-D	Equaliser	2
II-E	BPM Visualisation	2
III	Design and Assumption Decisions	2
III-A	Visual Interface and Cognitive Load	2
III-B	Operational Boundaries and Safety	3
III-C	Engineering Trade-off on BPM	3
IV	Unique Features and Behaviours	3
IV-A	Library Search	3
IV-B	Visual Cues for Audio Monitoring	3
IV-C	Application Icon	3
IV-D	Reset Equalizer	3
V	Results and Demonstration	4
VI	Conclusion	4

LIST OF TABLES

I	Results Summary	4
----------	----------------------------------	----------

A Report on DJ Application Development Using Object Oriented Programming Concepts

Abstract—This report details the development of *OtoDecks*, a DJ application developed in C++ using JUCE framework for the Object Oriented Programming module of the BSc Computer Science programme under the academic direction of Goldsmiths, University of London. It implements functionalities like real-time audio manipulation via IIR filters, dynamic BPM calculation and music library. This document reviews the implementation logic, design and assumption decisions, the results and demonstration, and finally the conclusion.

I. INTRODUCTION

The objective of this project was to design and implement *OtoDecks*, a digital DJ application. Developed as part of the Object-Oriented Programming (OOP) module for the BSc Computer Science at the University of London, it integrates principles of software engineering with object oriented programming.

Digital DJing software requires a high-performance environment capable of handling low-latency audio while providing an intuitive graphical user interface (GUI). In order to meet these requirements, C++ was selected due to its efficiency in memory management.

The development of *OtoDecks* focused on three core pillars:

- **Data Management**, developed as a music library and a hot cue system that allows users to save and retrieve track metadata across sessions.
- **User Interface (GUI)**, a dual-deck layout with waveform rendering and interactive controls.
- **Audio Engine**, implemented as a real-time playback for speed control, and a three-band equalizer.

II. IMPLEMENTATION LOGIC

The software architecture of *OtoDecks* follows a modular design, following the four pillars of Object Oriented Programming:

- **Encapsulation**: Groups data and methods into a single unit. Demonstrated by the `DJAudioPlayer` class, which sets audio data and methods.
- **Abstraction**: Hiding implementation details to show only essential features. The GUI components interact with complex audio processing through high-level methods, hiding the underlying DSP complexity from the user interface logic.
- **Inheritance**: Deriving new classes from existing ones. Extensive use of the JUCE framework involved inheriting from base classes such as `AudioSource`, `Button::Listener`, and `Slider::Listener` to extend their functionalities.

- **Polymorphism**: Using a single interface for different forms. Implemented through method overriding, like in the `TableListBoxModel` and listener callbacks.

The project was organized to manage the complexity of the application, separating classes and components, which helps structure dependencies in large systems (Sangwan, Vercellone-Smith, and Laplante 2008). To keep the codebase understandable, the implementation was divided into header and source files, and according to its functionalities, following a separation of concerns (SoC).

A. Loading, Playback and Mixing

The audio engine was encapsulated in `DJAudioPlayer` class, where it manages the playback acting as a discrete element with defined interfaces (ISO/IEC/IEEE 2010), which ensured that components have clear, and defined boundaries.

For loading, the tracks are initialized using `AudioFormatReaderSource`, where a null pointer was implemented to avoid runtime crashes and to ensure stability:

```
auto* reader = formatManager.createReaderFor(
    audioURL.createInputStream(false));
if (reader != nullptr) { // Safety check
    against unsupported formats
    std::unique_ptr<AudioFormatReaderSource>
        newSource (new AudioFormatReaderSource
            (reader, true));
    transportSource.setSource (newSource.get()
        , 0, nullptr, reader->sampleRate);
}
```

Moreover, for mixing and speed, a volume and playback speed were implemented, controlled via `setGain` and `setResamplingRatio`. A conditional colour changing logic was implemented within the `sliderValueChanged` listener to update the `thumbColourId` and `trackColourId` based on the double value of the slider:

```
if (val > 0.8) {
    slider->setColour(juce::Slider::
        thumbColourId, juce::Colours::red);
} else if (val > 0.5) {
    slider->setColour(juce::Slider::
        thumbColourId, juce::Colours::orange);
} else {
    slider->setColour(juce::Slider::
        thumbColourId, juce::Colours::green);
}
```

B. Music Library and Persistence

The library utilises a `TableListBoxModel` to manage track metadata.

To simulate a connection and track the users status across sessions, the application uses state data stored locally (OWASP Foundation 2010). The persistence mechanism formats file paths from the `tracks` vector, creating a structure that is easily retrievable (Sangwan, Vercellone-Smith, and Laplante 2008):

```
for (const auto& track : tracks) {
    juce::String filePath = track.url.
        getLocalFile().getFullPathName();
    output.writeText(filePath + "\n", false,
        false, nullptr);
}
```

C. Interactive Hot Cues

The Hot Cue system allows DJs to save and trigger specific time points instantly from the music in both decks.

In order to implement the cues, an array `cuePositions[8]` stores the relative positions. The implementation makes use of `ModifierKeys` to allow the user to modify or even overwrite individual cues (if needed):

```
if (cuePositions[i] == -1.0 || juce::
    ModifierKeys::getCurrentModifiers().
    isShiftDown()) {
    cuePositions[i] = player->
        getPositionRelative();
    saveCues();
}
```

D. Equaliser

The implementation of the equaliser using DSP Module was made by developing a function `ProcessorChain` with three knobs to allow the user to adjust the equaliser value of a track to high, medium or low.

The persistence could be seen since the equaliser settings are saved into a `.eq` file, making the user's mixing environment preserved through the handling of parameters and state data (OWASP Foundation 2010).

E. BPM Visualisation

Beats Per Minute (BPM) is the standard measure of musical beats that occur within one minute. Beyond the DJ booth, BPM is a metric in various fields, such as medicine, where a constant rhythm of 100 BPM is utilised as a gold standard for maintaining the correct compression rate during cardiopulmonary resuscitation (CPR) (Kim et al. 2017). Within a DJ application, BPM has two main purposes:

- **Beatmatching**, to align the BPM of two different tracks so they play at the same speed.
- **Library Organisation**, to allow the DJ to select tracks with compatible tempos.

In popular DJ applications, such as Serato or Rekordbox, it offers advanced features beyond a simple BPM display:

- Automatically matches the BPM and phase of two decks.
- Visual markers placed over the waveform that allow a looping and effects synchronisation.

Conceptually, BPM can be calculated by detecting sudden peaks in audio energy, which are usually drum hits. By measuring the time interval between these peaks over a specific window, the tempo is derived using the formula: $BPM = 60 / \text{average interval in seconds}$. This process often involves Fast Fourier Transform (FFT) analysis to isolate low frequency kicks from the rest of the signal. Instead, a metadata approach combined with real time recalculation was selected.

During development, the BPM display was implemented using a metadata approach. Hence, the BPM creation was calculated using the following formula: $\text{Live BPM} = \text{Base BPM} \times \text{Playback Speed Ratio}$, where the regular BPM was set to 120.

In the code, `bpmLabel` in the `DeckGUI` is updated whenever the `speedSlider` is manipulated. This calculation treats it as a dynamic state data that tracks the status of the playback session:

```
double baseBpm = player->getBpm(); // Call to
    public getter
double liveBpm = baseBpm * speedVal; //
    Recalculated live value based on R1D
bpmLabel.setText("BPM: " + juce::String(
    liveBpm, 1), juce::dontSendNotification);
```

III. DESIGN AND ASSUMPTION DECISIONS

In this section, it will be discussed assumptions, decisions, or unique features and behaviors of the application.

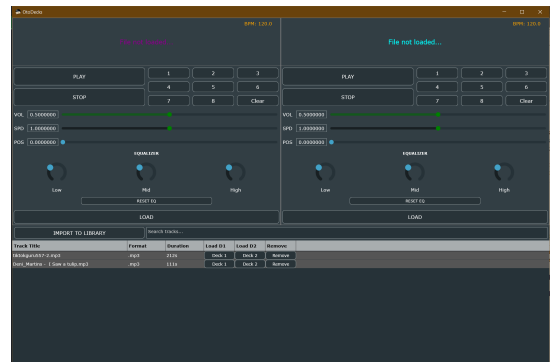


Fig. 1. Main Layout

A. Visual Interface and Cognitive Load

The first design decision was to prioritise a clean, and mirrored layout, keeping two decks side by side. The assumption was that in a live environment, a DJ requires visual consistency to avoid operational errors.

B. Operational Boundaries and Safety

To ensure the stability of the audio engine, operational boundaries was implemented for the playback speed, where the slider was capped between 0.0x and 2.0x, assuming that values outside this range would result in a bad experience or even audio instability.

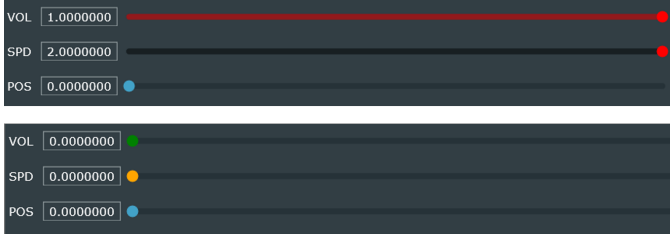


Fig. 2. Max and Min Speed

C. Engineering Trade-off on BPM

It was assumed that implementing a detection algorithm (FFT-based transient analysis) from scratch would be complex given the project deadline. It would have required several tests to ensure accuracy, which was not feasible within the project scope, considering the Agile triangle constraints (value, quality, and constraints). (Beck et al. 2001).

Instead, the BPM implementation was calculated using the following formula: Live BPM = Base BPM x Playback Speed Ratio, where the regular BPM was set to 120.

The display of the BPM was set to one decimal point (e.g., 124.5), assuming a balance between technical precision and visual clarity, preventing the GUI from being over complicated.

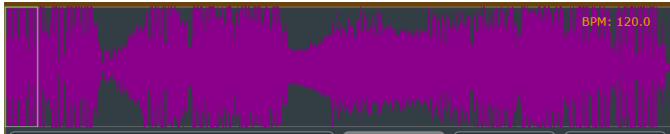


Fig. 3. BPM Display

IV. UNIQUE FEATURES AND BEHAVIOURS

Beyond the core requirements, OtoDecks includes specific features designed to improve the professional user experience.

A. Library Search

When DJs play in big parties, large music libraries are common, therefore manual scrolling would take too long if the user needs to rapidly select a music to keep the set. Hence, to avoid this issue and improve usability, a **Search Bar** feature was implemented. It provides real-time "as-you-type" filtering, focusing on delivering value to the user (Beck et al. 2001).

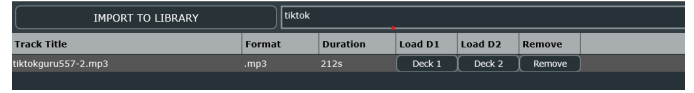


Fig. 4. Search Bar

B. Visual Cues for Audio Monitoring

To provide immediate feedback without requiring the user to read precise decimal values, the implementation of a three colour coding system for the volume sliders where the colours from the bar update (Green → Orange → Red) as slides to the left or to the right, assuming that DJs need a warning system to prevent clipping.



Fig. 5. Traffic Light Volume

C. Application Icon

A special icon was developed, including both **Large** and **Small icons** as required in JUCE framework for the application, this decision ensures that the application is easily identifiable within the taskbar and desktop environments.

The visual assets for these icons were generated using Artificial Intelligence (AI) tools.



Fig. 6. Icon

D. Reset Equalizer

Since changing the equalizer can be challenging and may confuse users when trying different types of music, a reset equalizer button was added. When clicked, it restores the audio to its default settings.

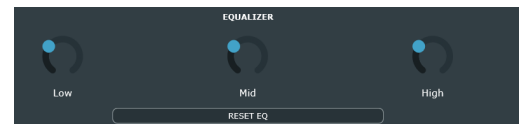


Fig. 7. Equalizer Reset Button

V. RESULTS AND DEMONSTRATION

The final implementation of OtoDecks was verified against the functional requirements to ensure system stability and performance. The application demonstrated reliable real-time audio manipulation and metadata persistence.

Additionally, a proper documentation is delivered together with the code, using Doxygen in all methods created in header files, using the following standard:

```
/**
 * @brief A one-sentence summary of what the
 *        function does.
 * @param Describes a specific input parameter
 *        (the variable name followed by its
 *        purpose).
 * @return Explains what the function sends
 *        back and under what conditions.
 */
```

TABLE I. RESULTS SUMMARY

Functionality	Status	Description
Playback Control	Fully Implemented	Play, Pause, Volume, and Speed
Music Library	Fully Implemented	Playlist with metadata persistence
Hot Cue System	Fully Implemented	Eight trigger buttons per deck
Three-Band Filter	Fully Implemented	IIR High, Mid, and Low EQ
BPM Logic	Fully Implemented	Metadata extraction and live display (with deterministic calculation if no metadata available)
(Unique Feature) Search Bar	Fully Implemented	Real-time "as-you-type" filtering
(Unique Feature) Custom Icons	Fully Implemented	Large and small icons
(Unique Feature) Visual Cues	Fully Implemented	Traffic light slider colours
(Unique Feature) Reset EQ.	Fully Implemented	Reset equalizer button

VI. CONCLUSION

The development of OtoDecks was a challenging project into the main principles of Object-Oriented Programming.

The most significant technical was understanding memory management and the use of pointers with C++ programming language. Understanding how to avoid memory leaks was initially hard at first. To overcome this, an additional C++ course on Udemy was taken, which provided the necessary knowledge to properly manage and develop, for example, smart pointers.

Learning how audio software operates "under the hood" challenging, and moving beyond high-level abstractions to understand how buffers works. This project allowed to apply theoretical engineering concepts to the DJ application.

If more development time were provided, I would aim to implement a "real" BPM detection system using Fast Fourier

Transform (FFT) analysis. However, the current version of OtoDecks stands as a scalable DJ tool that fulfills all the requirements for the final project.

REFERENCES

- Beck, K. et al. (2001). *Manifesto for Agile Software Development*. URL: <https://agilemanifesto.org/> (visited on 03/07/2026).
- ISO/IEC/IEEE (2010). *ISO/IEC/IEEE 24765:2010(E) - Systems and Software Engineering: Vocabulary*. 1st. Geneva, Switzerland: ISO.
- Kim, K. W. et al. (2017). "Effectiveness of 100 Beats per Minute Music on Cardiopulmonary Resuscitation Compression Rate Education: A Manikin Study". In: *The Journal of Emergency Medicine* 52.1.
- OWASP Foundation (2010). *OWASP Secure Coding Practices - Quick Reference Guide*. Version 2.0.1. URL: <https://owasp.org/> (visited on 03/07/2026).
- Sangwan, R. S., P. Vercellone-Smith, and P. A. Laplante (2008). "Structural Epochs in the Complexity of Software over Time". In: *IEEE Software* 25.4.